

Chapitre 1 Introduction

1.1 Symfony est un Framework PHP (Cadre de travail).

Il va donc vous offrir un cadre de travail. Contrairement au cours de PHP où l'on devait créer tous nous même, il vous fournira un ensemble de classe que vous allez pouvoir utiliser pour créer une application WEB plus facilement. Il va vous donner une structure pour organiser votre projet.

C'est un Framework qui utilise la structure MVC (Model View Controller) qui est une manière de séparer son code plus proprement. Le **Model** c'est un peu comme l'administrateur, tout ce qui est source de données (la communauté de Symfony utilise doctrine). La **View**/vue (gérer généralement en **twig** qui est un moteur de Template) c'est un peu comme le marketing, tout ce qui est affichage. Le **Controller** c'est un peu comme le CEO de l'entreprise, c'est lui qui donne les directives, afin de dire au model récupère moi tel information en base de données et ensuite dire à la vue d'afficher l'information. C'est le chef d'orchestre. Le MVC se retrouve dans de nombreux Frameworks.

A l'heure actuelle où je crée ce cours, nous venons de passer à la version 7 de Symfony. J'ai fait un cours avec la version 5 et 6, sachez qu'il n'y a pas forcément de gros changement majeur. Il y a eu quelques petits changements entre la version 5 et 6 de Symfony parce que PHP passait de la version 7 à 8. Ce qu'on verra dans ce cours sera transférable sur la version 8, 9 etc. de Symfony.

Symfony 7 utilise PHP à partir de la version 8.2 donc pour pouvoir travailler dessus il faut vérifier que vous avez une version PHP ≥ 8.2 .

1.2 Programme

Nous débuterons en explorant le Framework Symfony. Nous installerons Symfony, examinerons sa structure et créerons nos premières pages pour comprendre son fonctionnement de base. Ensuite, nous aborderons des concepts plus avancés à travers des exercices pratiques. Au cours de cette formation, nous élaborerons un petit site pour gérer des recettes de cuisine. Par la suite, nous plongerons dans des sujets plus spécifiques, tels que le fonctionnement interne, les systèmes de permission, etc. Enfin, nous approfondirons nos connaissances avec des travaux pratiques sur des cas concrets.

À ce stade, vous devriez être capable d'utiliser Symfony de manière autonome et d'explorer de nouveaux concepts par vous-même.

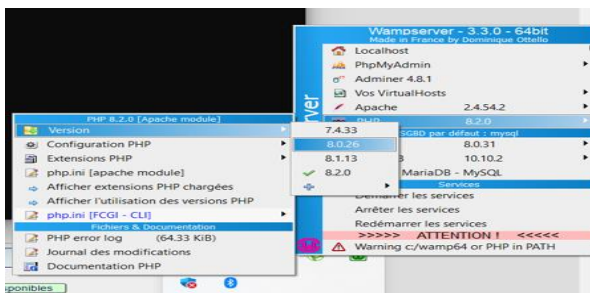
Pour pouvoir bien suivre ce cours, il est essentiel de maîtriser PHP. Si les termes comme "classe", "propriété publique ou privée", ou "instance" ne vous sont pas familiers, il serait difficile de comprendre les concepts avancés de Symfony. Il est aussi essentiel de maîtriser SQL, que les notions de tables, relations, clé primaire etc. soient acquise. De plus, vous devriez être à l'aise avec l'utilisation du terminal pour exécuter des commandes (CMD), car nous en aurons besoin pour interagir avec le Framework, par exemple pour générer du code. Enfin nous verrons un peu Composer, car cela vous aidera à installer le Framework et à ajouter de nouvelles dépendances.

Chapitre 2 Installation et découverte

2.1 Installation

Dans ce chapitre nous allons installer et découvrir la structure du Framework Symfony. Pour avoir plus d'information sur Symfony, il existe le site officiel qui est la documentation principale du Framework : www.symfony.com. Comme pour mon cours de PHP (php.net) et de MySQL(sql.sh), je vous recommande de mettre ce site en favoris, comme ça si vous avez besoin d'informations sur le Framework, vous y aurez accès directement.

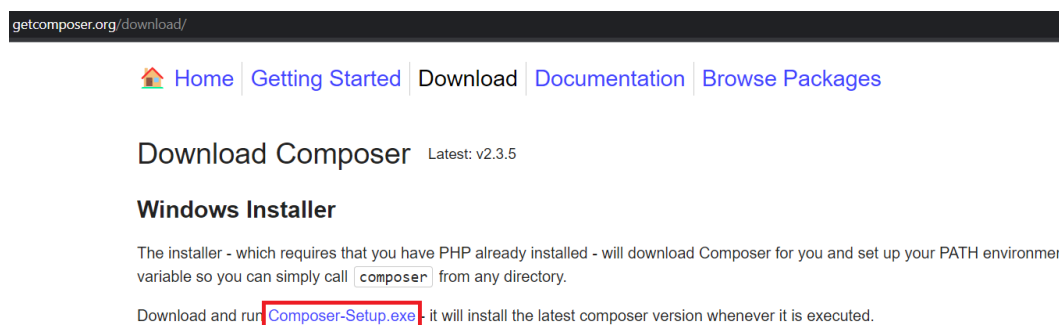
Allons sur le site dans documentation Symfony Doc, ensuite dans getting started Setup et Installation. Vérifier tout d'abord que vous avez bien une version 8.2 de PHP ou supérieur dans le terminal en tapant « **php -v** ». Ceux qui ont une ancienne version, je vous invite d'abord à regarder dans Wamp quelle version de PHP sont disponible :



Si dans Wamp vous n'avez que des versions inférieures à PHP 8.2, je vous invite à mettre à jour votre WampServer, pour pouvoir avoir les dernières versions de PHP en allant sur : <https://wampserver.aviatechno.net/?lang=fr> lors de l'installation de la mise à jour de Wamp n'oubliez pas de prendre les Microsoft Visual C++ à partir de 2010 dans les 2 versions x86 et x64. Une fois que votre WampServer est à jour, vous pourrez directement choisir votre version de php que vous voulez installer sur ce site. ATTENTION, une fois fini il faut modifier vos variables d'environnements système, pour choisir cette nouvelle version de PHP.

On va aussi installer **composer**. Il va nous permettre de gérer toutes les dépendances de notre projet, donc tous les packages dont le projet va dépendre. Je l'expliquerai par après.

Allez sur <https://getcomposer.org/download/> télécharger et installer « **composer-setup.exe** » :

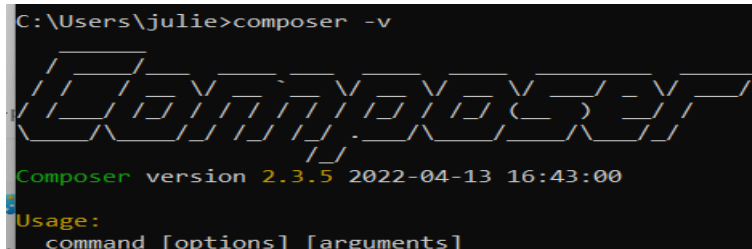


Il n'y a rien à cocher lors de l'installation.

Une fois fini lancer votre CMD et taper la commande :

- **composer -v**

Cette commande, vous donnera la version de « **composer** » sur votre machine.



```
C:\Users\julie>composer -v
Composer version 2.3.5 2022-04-13 16:43:00
Usage:
command [options] [arguments]
```

Composer est installé maintenant il suffit d'installer Symfony. Allez sur <https://symfony.com/download> pour voir comment installer sur LINUX, MAC et Windows. Je vais montrer comment le faire sur Windows. Les personnes sur Linux et Mac, il suffit de suivre le tuto sur le site.

Une fois que c'est fait nous allons installer « **Scoop** » qui est un programme d'installation en ligne de commande pour Windows.

Tapez dans la barre de recherche Windows « **PowerShell** » et exécuter. Dans le terminal PowerShell taper cette commande qui va installer scoop :

- **Set-ExecutionPolicy** -ExecutionPolicy RemoteSigned -Scope CurrentUser
- **Invoke-RestMethod** -Uri <https://get.scoop.sh> | **Invoke-Expression**

Une fois installé, **quittez PowerShell** et lancer votre **CMD** (ligne de commande Windows).

Taper « **scoop** » pour voir s'il est bien installé. Si c'est le cas maintenant on va installer **symfony-cli** grâce à cette commande :

- **scoop install symfony-cli**

A partir d'ici les personnes sous Linux et Mac qui ont fini les installations vous pouvez nous rejoindre et vérifier que vous avez Symfony CLI installé sur vos machines en tapant dans la console (cmd) :

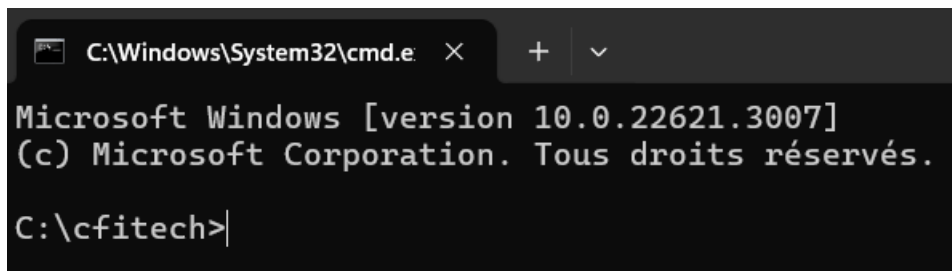
- **symfony -v** (ici il va vérifier la version)

J'ai décidé de passer par symfony cli en ligne de commande. Il faut savoir qu'il est possible de lancer symfony en passant par composer.

2.2 Création d'un projet

Maintenant que nous avons l'environnement installé de symfony, nous allons créer un nouveau projet. Je vous invite à aller dans votre PC et créer un projet à la **racine C:**. Vous allez créer un nouveau dossier « **cfitech** » et entrer dedans.

Vous lancerez le cmd dans ce dossier.



```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [version 10.0.22621.3007]
(c) Microsoft Corporation. Tous droits réservés.
C:\cfitech>
```

Et là nous allons créer notre premier projet Symfony en tapant la commande :

- `symfony new TutoSymfony`

Ici on va donc créer un nouveau projet qui s'appelle « **TutoSymfony** ». A noter que je n'ai pas précisé la version de symfony comme c'est fait sur la doc officiel.

Cette commande va automatiquement créer le dossier correspondant et installer les différentes dépendances qui sont nécessaires. Il faudra ensuite rentrer dans ce dossier avec la commande « **cd TutoSymfony** ». Une fois dedans vous pouvez taper « **code .** » pour ouvrir Visual Studio Code.

Avec la commande qu'on a tapée, on a installé le squelette c'est-à-dire une version de symfony qui est ultra légère et qui ne contient aucun des modules qui va être nécessaire à la création d'une application WEB. On n'a pas par exemple de système pour générer des vues HTML, on n'a pas la partie connexion à la base de données etc. Avec cette commande on devra rajouter par après les dépendances dont on a besoin pour créer une applications WEB.

On va donc ajouter une commande en plus qui va justement installer tous ceux dont on aura besoin pour une application WEB et c'est via composer qu'on peut le faire.

Composer va nous permettre de gérer toutes les dépendances de notre projet, donc tous les packages dont le projet va dépendre. Quand elles sont installées, elles vont se retrouver au niveau du dossier **vendor**. Donc à chaque fois qu'on fait « **composer require nomDuComposant** » on pourra le retrouver dans vendor.

A noter qu'on peut écrire :

- **composer req** comme raccourci pour composer **require**
- **composer rem** comme raccourci pour composer **remove**

On va donc installer webapp dans notre projet symfony en tapant :

- **composer require webapp**

Ça va installer tout un tas de dépendances supplémentaire. Il va nous demander si on veut inclure une configuration docker, et on dira oui. Normalement nous verrons docker beaucoup plus tard dans le cadre de votre formation.

A noter que si vous voulez on peut directement créer un projet symfony complet pour une application web en mettant directement dans la commande le webapp, donc dans C:\cfitech :

- **symfony new TutoSymfony --webapp**

C'est cette commande que je vous recommande de faire à chaque création de nouveau projet symfony. Après le faire de l'autre manière c'est totalement pareil.

Pour vérifier la version php de Symfony :

- **symfony php -v**

Pour vérifier la liste de version php prise en charge par Symfony :

- **symfony local:php:list**

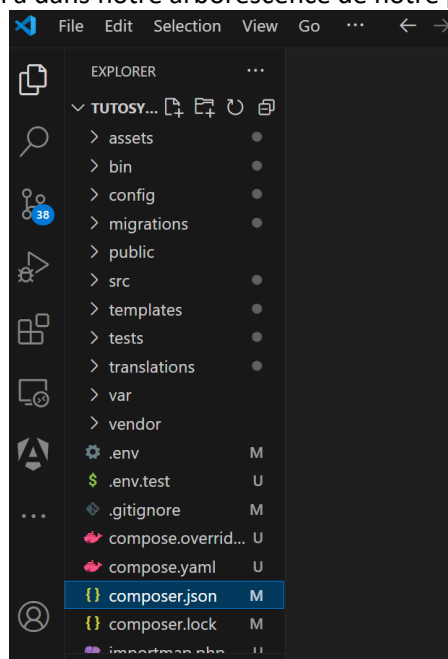
Pour changer la version de php dans le projet Symfony :

- **echo 8.*.* > .php-version**

Vous avez remarqué qu'il y a énormément de fichier dans votre projet, on va un peu les parcourir.

2.3 Description des dossiers et fichiers

On va un peu voir tous ce qu'on a dans notre arborescence de notre projet Symfony.



Le répertoire **assets/** contient des fichiers javascript et CSS.

Le répertoire **bin/** contient le principal point d'entrée de la ligne de commande : console. Vous l'utiliserez tout le temps. Il contient un deuxième fichier qui est phpunit, qui est une sorte de pont avec l'outil de test.

Le répertoire **config/** est constitué d'un ensemble de fichiers de configuration sensibles, initialisés avec des valeurs par défaut. Un fichier par paquet. Vous les modifierez de temps en temps. Ça utilise la syntaxe **yaml** qui est basé sur l'utilisation de l'indentation comme en python.

Le répertoire **migrations/** qui va contenir les migrations, qui sera très utilisé pour les informations concernant nos migrations en base de données.

Le répertoire **public/** est le répertoire racine du site web, et le script index.php est le point d'entrée principal de toutes les ressources HTTP dynamiques. Quand on va héberger notre application Symfony, c'est ce dossier la qui servira de racine pour notre serveur Web.

Le répertoire **src/** héberge tout le code (nos classes) que vous allez écrire ; **c'est ici que vous passerez la plupart de votre temps.** Par défaut, toutes les classes de ce répertoire utilisent le *namespace* App. C'est votre répertoire de travail, votre code, votre logique de domaine. Symfony n'a pas grand-chose à y faire. Dans le fichier composer.json à la racine du projet, si vous l'ouvrez, vous pourrez voir dans autoload qu'on a le namespace APP qui est branché sur le dossier src.

Le répertoire **templates/** va contenir nos vues HTML qui utilisent une extension twig.

Le répertoire **tests/** qui permettra d'écrire nos tests.

Le répertoire **translations/** va contenir les traductions parce que Symfony va nous permettre de créer un site multi langues.

Le répertoire **var/** contient les fichiers temporaires, les logs et les fichiers générés par l'application lors de son exécution. Vous pouvez le laisser tranquille. C'est le seul répertoire qui doit être en écriture en production.

Le répertoire **vendor/** contient tous les paquets installés par Composer, y compris Symfony lui-même. C'est notre arme secrète pour un maximum de productivité. Ne réinventons pas la roue. Vous profiterez des bibliothèques existantes pour vous faciliter le travail. Le répertoire est géré par Composer. N'y touchez jamais.

Le fichier **.env** va surtout permettre de gérer nos variables d'environnements. Il va nous permettre de configurer un peu le Framework. C'est ici qu'on configurera la connexion à notre base de données.

Le fichier **.gitignore** qui est en rapport avec git, pour indiquer les fichier qu' il ne faudra pas tracker.

Le fichier **composer.json** c'est le fichier de configuration de composer.

Le fichier **composer.lock** va nous permettre de locker spécifiquement les versions qui ont été installé.

Le fichier **symfony.lock** qui est en rapport avec Symfony Flex, la manière de gérer les applications Symfony.

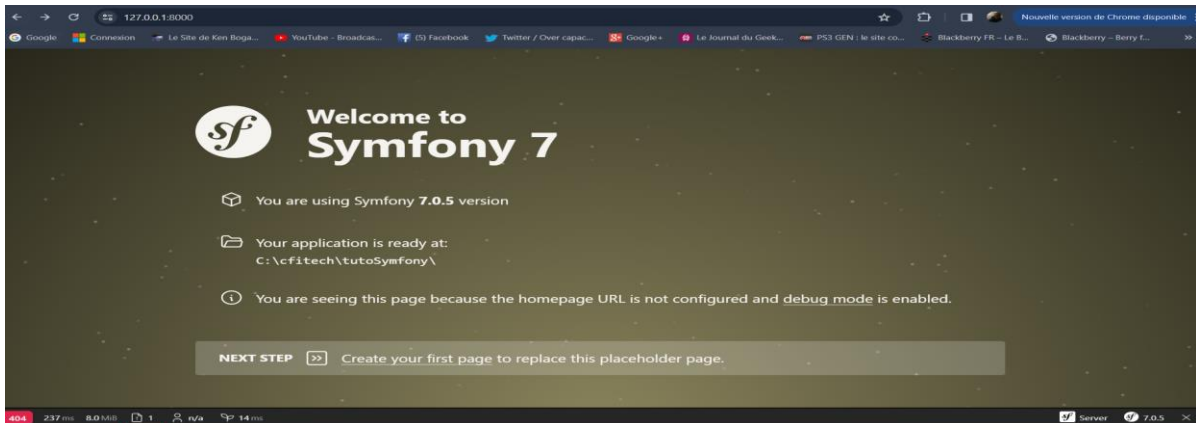
Voilà une petite explication grosso modo des dossiers et fichiers qui composent notre projet. Je vous ai dit qu'il y a plus de 9000 fichiers et plus de 1000 dossiers lorsque le projet est créé avec webapp.

2.4 Lancer notre serveur

Si vous êtes toujours dans le dossier de votre projet dans la console. On va lancer le serveur :

- `symfony server:start`

Votre serveur est lancé et vous pouvez aller sur votre navigateur et taper 127.0.0.1 :8000 et normalement vous tomberez sur une page comme ça :



C'est une page qui est automatiquement générée par Symfony et qui nous présente les premières étapes à faire. En bas de la page nous avons un outil hyper important qui est une debug bar. Elle est activée par défaut quand on est en mode DEV. On l'utilisera pour déboguer justement. Il permet de voir aussi sous quelle version de Symfony tourne votre application (en bas à droite).

Il faut savoir qu'on peut lancer le serveur en back grounds en ajoutant un « -d », on peut le faire via la commande :

- **`symfony server:start -d`** (ou le raccourci **`symfony serve -d`**)

Pour lancer automatiquement la page locale après avoir lancer le serveur on fait :

- **`symfony open:local`**

Pour arrêter le serveur on peut faire la commande :

- **`symfony server:stop`** (Si ça ne fonctionne pas faites **Ctrl+C** dans votre terminal)

Pour pouvoir voir tous les serveurs lancer :

- **`symfony server:list`**

Pour lancer un serveur Symfony en **HTTPS** (sécurisé) :

- **`symfony server:ca:install`** (accepter le certificat)
- **`symfony serve -d`**

A noter que vous pouvez lancer plusieurs projets en même temps, il allouera juste les serveurs en incrémentant de 1 à chaque fois ex : 8000, puis 8001, puis 8002 etc.